

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

ОТЧЕТ
Домашняя работа №4

Выполнила: Красноперова Е. Д.

К3443

Проверил: Добряков Д. И.

Санкт-Петербург,

2026 г.

Задача

1. Разделить монолитное бэкенд-приложение на микросервисы, придерживаясь концепции database-per-service.
2. Спроектировать микросервисную архитектуру: отразить взаимосвязь между микросервисами и описать способы их взаимодействия друг с другом.
3. Спроектировать разделение существующей базы данных на обособленные базы данных.
4. Спроектировать и описать эндпоинты, необходимые для межсервисного взаимодействия, в формате спецификации OpenAPI.
5. Описать варианты запросов и ответов, задокументировать возможные ошибки.
6. Сформировать документ с конкретными требованиями и шагами для перехода от монолитного бэкенд-приложения к отдельным микросервисам.

Ход работы

Проект представляет собой сервис для обмена рецептами и кулинарных блогов. Сервис поддерживает вход и регистрацию, личный кабинет с сохранёнными рецептами и публикациями, поиск и фильтрацию рецептов, страницу рецепта с фото, пошаговыми инструкциями и видео, а также комментарии, лайки и подписки на кулинаров.

В предыдущих работах для проекта выделены сущности User, Recipe, Step, Ingredient, RecipeIngredient, Comment, Like, SavedRecipe и Subscription. При переходе к микросервисной архитектуре границы сервисов выбраны по бизнес-областям. Каждая база данных принадлежит одному сервису, прямой доступ к чужой БД отсутствует.

1. Разделение монолита на микросервисы

UserService отвечает за регистрацию, вход, JWT-аутентификацию и профиль пользователя.

RecipeService отвечает за рецепты, ингредиенты, состав рецепта и пошаговые инструкции, а также за операции поиска и фильтрации рецептов.

SocialService отвечает за лайки, комментарии, сохранённые рецепты и подписки.

API Gateway является единой точкой входа для клиента и маршрутизирует запросы к соответствующим сервисам. Собственной бизнес-БД Gateway не имеет.

2. Микросервисная архитектура

Для синхронного межсервисного взаимодействия используется HTTP REST и JSON. Внутренние операции имеют префикс /internal. Межсервисные запросы защищаются сервисной авторизацией. Например, при добавлении лайка SocialService проверяет пользователя через UserService и рецепт через RecipeService, после чего записывает Like только в Social DB.

3. Разделение базы данных

Сервис	База данных	Сущности
UserService	User DB	User
RecipeService	Recipe DB	Recipe, Step, Ingredient, RecipeIngredient
SocialService	Social DB	Comment, Like, SavedRecipe, Subscription

В Social DB поля user_id и recipe_id хранят идентификаторы объектов других сервисов, но не являются внешними ключами между физическими базами. Их существование проверяется через HTTP API. Внутри Recipe DB сохраняются связи рецептов с шагами и ингредиентами; внутри Social DB - социальные связи.

4. Эндпоинты межсервисного взаимодействия

4.1 UserService

```
GET /internal/users/{user_id}
GET /internal/users/{user_id}/exists
```

Первый эндпоинт возвращает краткие данные пользователя. Второй используется для проверки существования пользователя.

4.2 RecipeService

```
GET /internal/recipes/{recipe_id}
GET /internal/recipes/{recipe_id}/exists
```

Эти эндпоинты используются SocialService перед созданием Like, Comment или SavedRecipe.

4.3 SocialService

```
POST /api/recipes/{recipe_id}/like
DELETE /api/recipes/{recipe_id}/like
POST /api/recipes/{recipe_id}/comments
GET /api/recipes/{recipe_id}/comments
POST /api/recipes/{recipe_id}/save
DELETE /api/recipes/{recipe_id}/save
POST /api/users/{user_id}/subscribe
DELETE /api/users/{user_id}/subscribe
```

Эти операции изменяют только Social DB. Для проверки связанных объектов SocialService обращается к UserService и RecipeService, а не к их базам напрямую.

5. OpenAPI для внутренних эндпоинтов

Используется OpenAPI 3.0.3. Ниже приведена спецификация основных внутренних операций.

```
openapi: 3.0.3
info:
  title: Internal Microservices API
  version: 1.0.0
paths:
  /internal/users/{user_id}:
    get:
      summary: Получить пользователя
      parameters:
        - name: user_id
          in: path
          required: true
          schema: {type: integer}
      responses:
        '200': {description: Пользователь найден}
        '401': {description: Ошибка сервисной авторизации}
        '404': {description: Пользователь не найден}
  /internal/users/{user_id}/exists:
    get:
      summary: Проверить существование пользователя
      parameters:
        - name: user_id
          in: path
          required: true
          schema: {type: integer}
```

```

    responses:
      '200': {description: Результат проверки}
      '401': {description: Ошибка сервисной авторизации}
/internal/recipes/{recipe_id}:
  get:
    summary: Получить рецепт
    parameters:
      - name: recipe_id
        in: path
        required: true
        schema: {type: integer}
    responses:
      '200': {description: Рецепт найден}
      '401': {description: Ошибка сервисной авторизации}
      '404': {description: Рецепт не найден}
/internal/recipes/{recipe_id}/exists:
  get:
    summary: Проверить существование рецепта
    parameters:
      - name: recipe_id
        in: path
        required: true
        schema: {type: integer}
    responses:
      '200': {description: Результат проверки}
      '401': {description: Ошибка сервисной авторизации}

```

6. Запросы, ответы и ошибки

6.1 Проверка пользователя

```

GET /internal/users/15
Authorization: Bearer <service-token>

200 OK
{
  "user_id": 15,
  "username": "elizaveta",
  "avatar_url": "https://example.com/avatar.jpg"
}

404 Not Found
{
  "error": {
    "code": "USER_NOT_FOUND",
    "message": "Пользователь не найден"
  }
}

```

6.2 Проверка рецепта

```

GET /internal/recipes/42/exists
Authorization: Bearer <service-token>

```

```

200 OK
{
  "recipe_id": 42,
  "exists": true
}

404 Not Found
{
  "error": {
    "code": "RECIPE_NOT_FOUND",
    "message": "Рецепт не найден"
  }
}

```

6.3 Лайк через SocialService

```

POST /api/recipes/42/like
Authorization: Bearer <user-jwt>

```

SocialService получает user_id из JWT, проверяет пользователя в UserService и рецепт в RecipeService, затем создаёт Like в Social DB.

```

200 OK
{
  "recipe_id": 42,
  "user_id": 15,
  "liked": true
}

409 Conflict
{
  "error": {
    "code": "ALREADY_LIKED",
    "message": "Рецепт уже отмечен пользователем"
  }
}

```

Для комментария, сохранения рецепта и подписки используется тот же принцип: проверка связанных объектов через владельцев данных, затем изменение Social DB.

7. Возможные ошибки

HTTP	Код	Описание
401	UNAUTHORIZED	Не пройдена авторизация пользователя или сервиса.
403	FORBIDDEN	Недостаточно прав.
404	USER_NOT_FOUND / RECIPE_NOT_FOUND	Связанный объект не найден.

409	ALREADY_LIKED / ALREADY_SAVED	Социальная связь уже существует.
422	VALIDATION_ERROR	Некорректные входные данные.
500	INTERNAL_ERROR	Внутренняя ошибка сервиса.
503	SERVICE_UNAVAILABLE	Зависимый сервис недоступен.

8. Требования и шаги перехода

1. Зафиксировать текущий публичный REST-контракт и поведение монолита.
2. Разделить доменные границы: пользователи - UserService; рецепты - RecipeService; социальные функции - SocialService.
3. Создать отдельные User DB, Recipe DB и Social DB и перенести принадлежащие им данные.
4. Убрать прямые внешние ключи и обращения к таблицам другого сервиса; междоменные связи хранить через идентификаторы.
5. Реализовать внутренние API UserService и RecipeService и единый формат ошибок.
6. Вынести социальные операции в SocialService и заменить обращения к чужим таблицам HTTP-вызовами.
7. Добавить API Gateway и настроить маршрутизацию.
8. Настроить JWT для пользовательских запросов и сервисную авторизацию для внутренних запросов.
9. Проверить интеграционные сценарии: регистрация - вход - просмотр рецепта - лайк/комментарий/сохранение - подписка.
10. После успешного тестирования отключить старые монолитные обработчики и завершить миграцию.

Вывод

Монолитный backend сервиса обмена рецептами и кулинарных блогов разделён на UserService, RecipeService и SocialService с API Gateway в качестве

единой точки входа. Для каждого сервиса определена собственная база данных, что соответствует database-per-service.